Module 1: Introduction



What is Data?

- Data refers to raw facts or observations (numerical, textual or symbolic).
- Data by itself may not convey complete meaning.
- Example: 75 (This is simply a value)
    - 75 what?
        - Marks?
        - Age?
        - Temperature?
        - Percentage?

Therefore, data generally needs context.

What is Information?

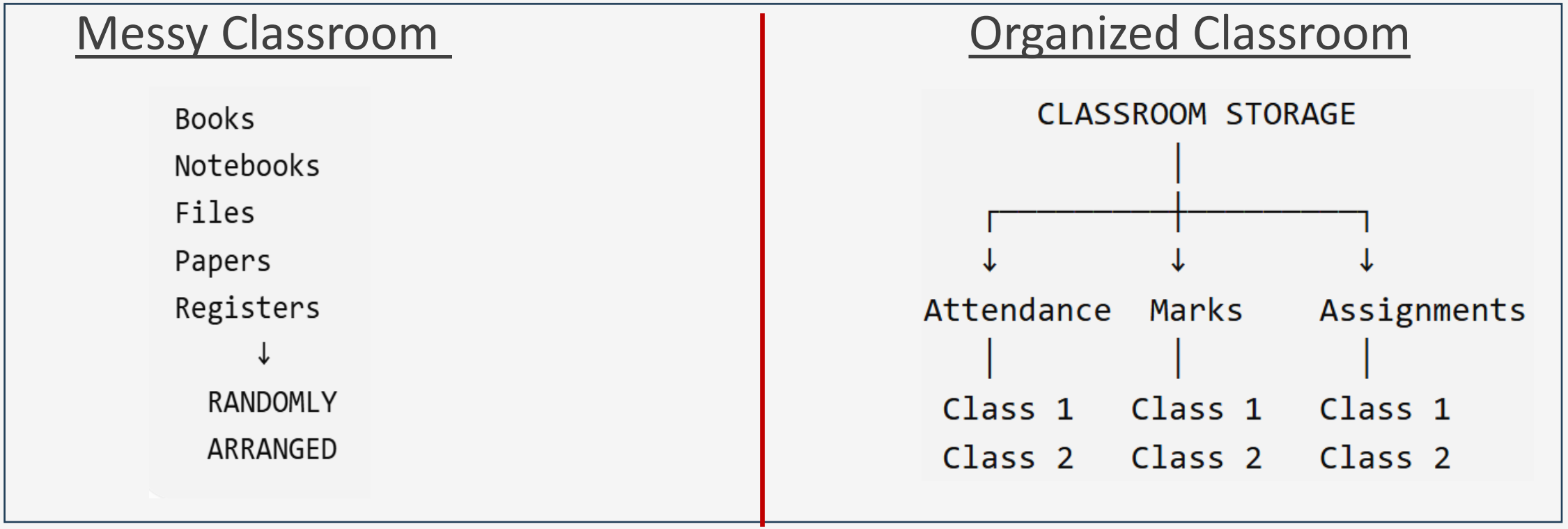
- **Information** is meaningful data obtained after processing or interpreting data.
- Example:

```
Marks
75  82  91  68  84
        ↓
    Processing
        ↓
Average = 80
Highest = 91
Lowest = 68
```

- Now the data has meaningful interpretation.
- Data → Information
    - **Data** = Raw facts
    - **Information** = Meaningful result obtained after processing data

What is Data Organization?

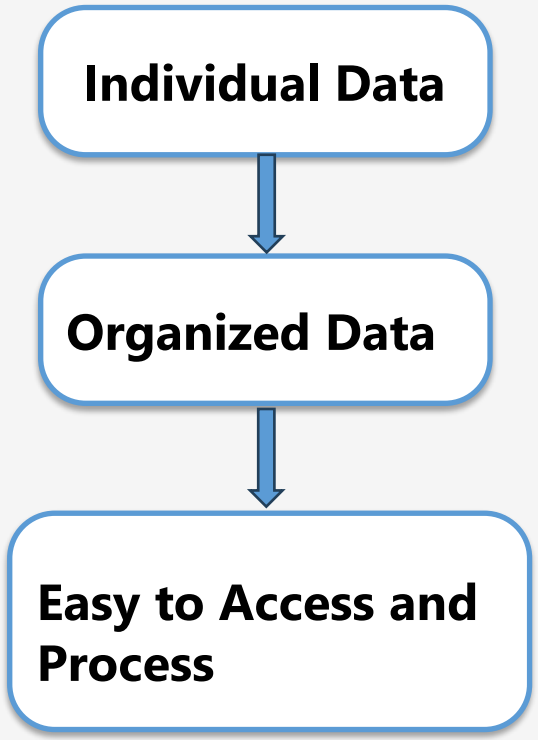
- **Why Data Organization?**
  - Example: Imagine



  - Therefore, Data must be organized so that it can be stored, accessed and processed efficiently.

Data Organization?

- Data organization is the systematic arrangement of data so that it can be
  - Easily Accessed
  - Easily Processed
  - Easily stored and updated
  - Quickly searched
  - Properly Maintained



Elementary Data Organization?

- Data can be viewed at different levels.

  Smallest → Largest

  Bit → Byte → Character → Field → Record → File → Database

- Elementary data organization explains how data is organized from its **smallest units to larger collections**.

- It refers to the **basic way of organizing and representing data** in a computer system. It starts with simple units such as **bits, bytes, characters, fields, and records**. These basic units are combined step-by-step to form **files and databases which** helps us understand how data is **stored, organized, and accessed** in a computer.

Elementary Data Organization

- **Bit**
  - It is the smallest unit of data in a computer.
  - A bit can have only two values: **0=OFF or 1=ON.**
  - **Example:** 101101 (6 bits).
- **Byte**
  - A **byte** is a unit of storage and it consists of 8 bits.
  - Example: 01000001 (8 bits and represents a byte).

Elementary Data Organization

- **Character**
  - A **character** is a symbol used to represent text.
  - Characters are represented internally using encoding schemes such as **ASCII** or **Unicode**.
  - **Example:** A can be represented in ASCII as: 01000001.
- **Field**
  - A **field** is a named data item that represents a particular attribute or property.
  - **Example: Student Registration Form**

Roll No.	Name	Age	Marks	⟶ **Fields**
10	Isha	20	85	

Elementary Data Organization

- **Record**
  - A **record** is a collection of related fields describing one entity.
  - **Example:** For student Isha

Field	Value
Roll No.	10
Name	Isha
Course	MCA
Marks	85

Together, these fields form **one student record**.

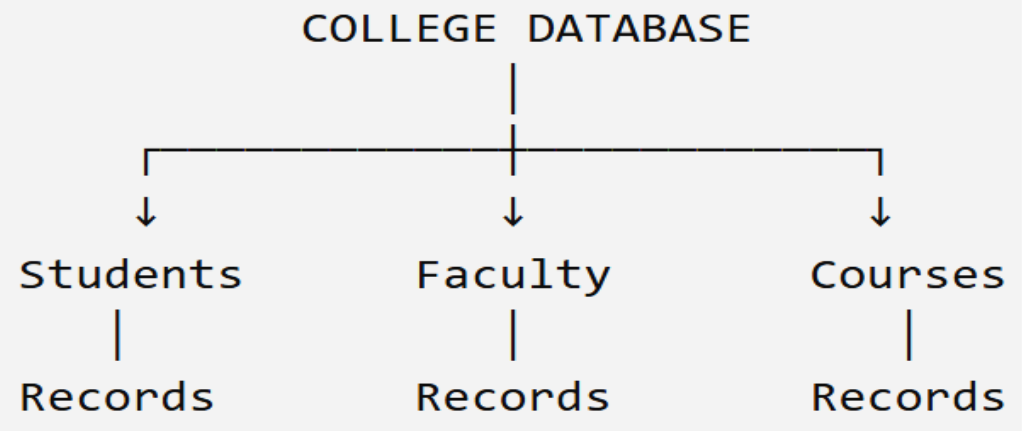
Elementary Data Organization

- **File**
  - A **file** is a collection of related records stored together.
  - **Example:** Suppose we have:
    - Student 1 → Record
    - Student 2 → Record
    - Student 3 → Record
    - Student 4 → Record

Together, these fields form **one student file** i.e. multiple student records form a file.

Elementary Data Organization

- **Database**
  - A **database** is an organized collection of related files.
    - A college database may contain:
      - Student information, Faculty information, Course information, Examination information, Fee information
- **Example: College Database**



Elementary Data Organization

Example: College Management System

```
Bit
 ↓
Byte
 ↓
Character
 ↓
Field: Student Name
 ↓
Record: One Student
 ↓
File: All Students
 ↓
Database: Entire College
```

What is a Data Structure?

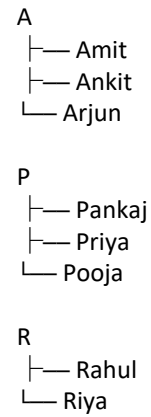
- A **data structure** is a systematic way of organizing and storing data in computer memory so that it can be accessed and modified efficiently by the application software.
- **Examples:** Array, Linked List, Stack, Queue, Tree, Graph
- **Real-life Scenario:** Imagine your phone has 2000 contacts

If the contacts are stored **randomly:**

Rahul
Amit
Priya
Zoya
Neha
...

❌ Finding a contact takes more time.

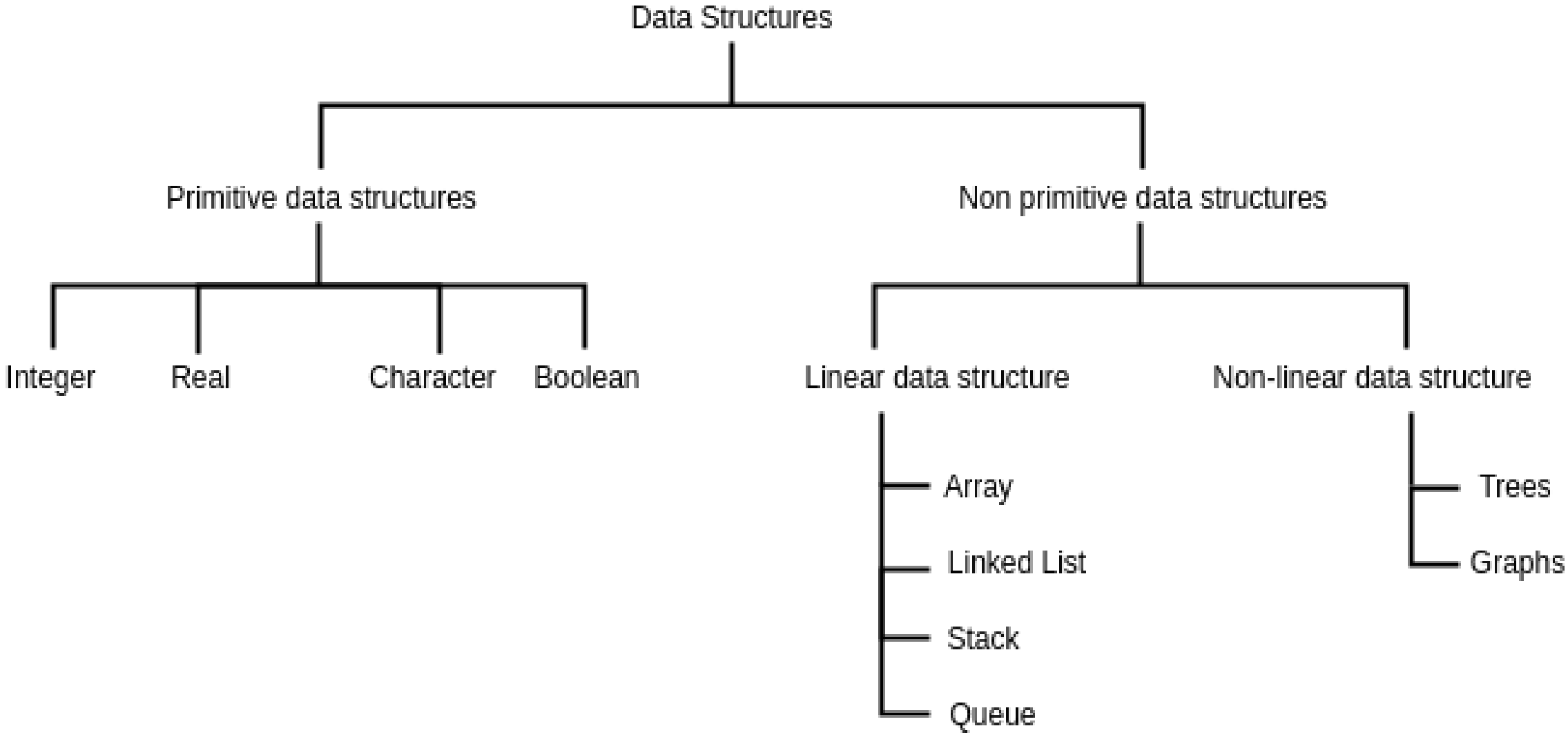
If the contacts are **organized:**



✅ Finding a contact becomes easier.

- Similarly, computers organize data using appropriate **data structures**.

Classification of Data Structures



Primitive vs Non-Primitive Data Structures

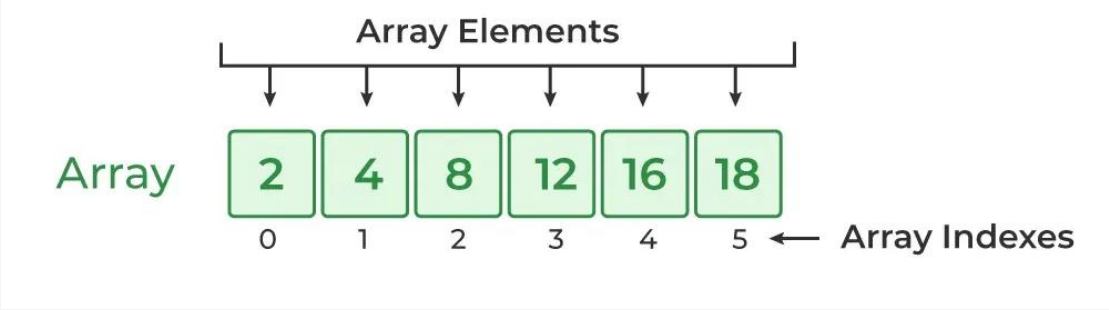
Primitive	**Non-Primitive**
Basic data types	More complex data structures
Generally stores a single value	Generally stores multiple values
Directly provided by programming languages	Usually constructed/implemented using basic types
Simple to use	More complex to organize and manipulate
Examples: int, char, float	Examples: Array, Stack, Queue, Linked List, Tree, Graph

Linear Data Structures

- A data structure is **linear** when:
  - all the elements are arranged in a linear or sequential order.
  - Example: A → B → C → D → E
  - Each element of the data structure has only one pre-decessor and one successor.
  - Example: Arrays, Stacks, Queues, Linked Lists

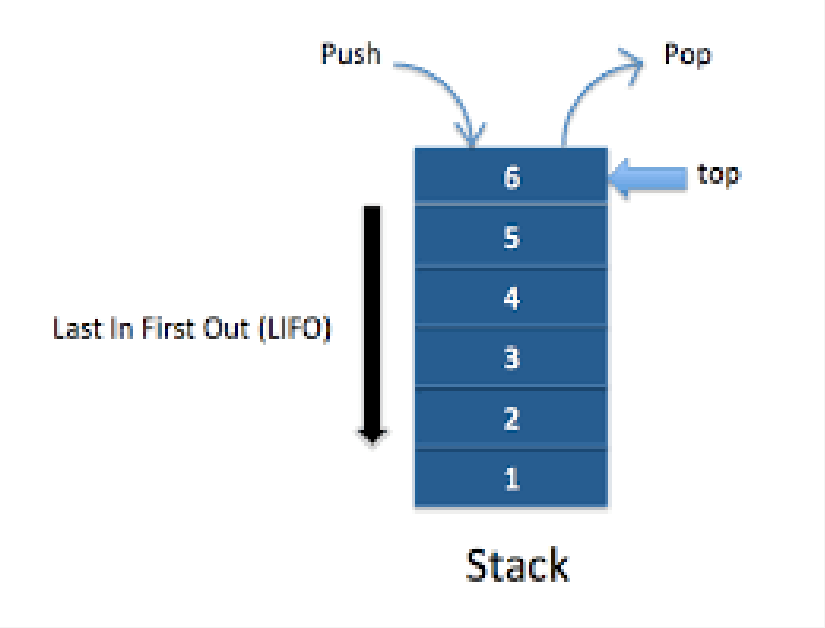
Linear Data Structures

- **Array**
  - Holds more than one element.
  - Holds only homogeneous elements.
  - It is index-based so accessing becomes faster.



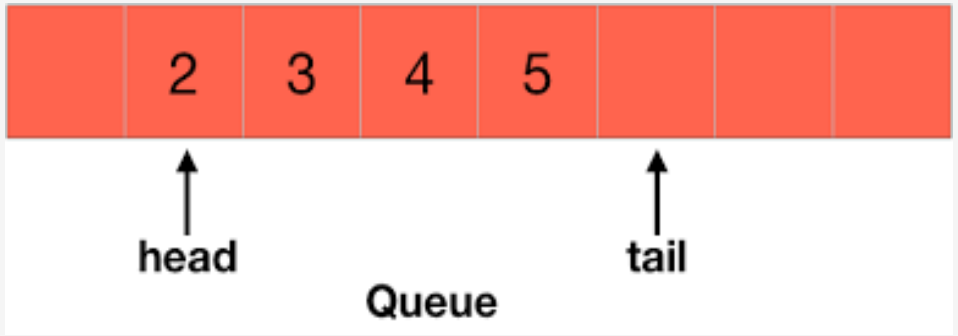
Linear Data Structures

- **Stack**
  - Data structure where insertion and deletion takes place at one end i.e. the TOP.
  - LIFO (Last In First Out) structure.
  - 2 basis operations
    - PUSH
    - POP



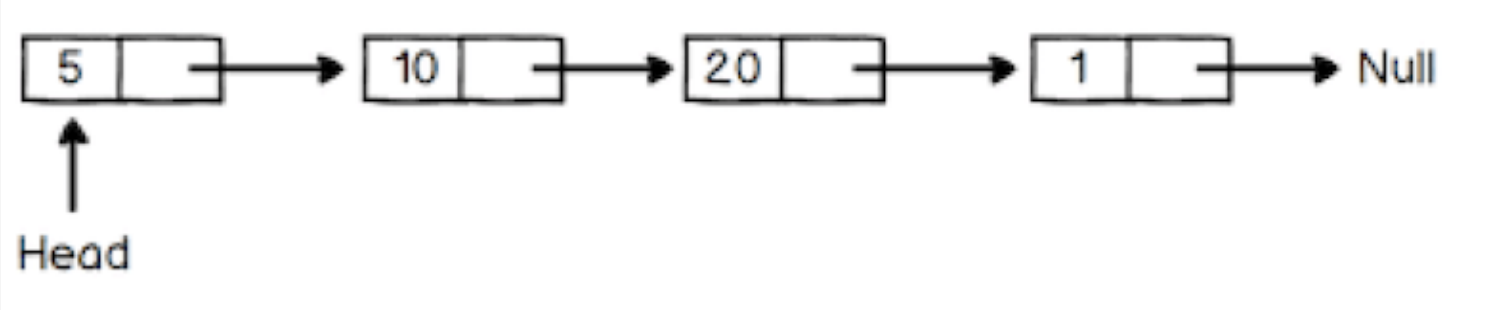
Linear Data Structures

- **Queue**
  - Linear list in which data can only be inserted at one end, called the *rear*, and deleted from the other end, called the *front*.
  - FIFO (First In First Out) structure.
  - 2 basis operations
    - Enqueue
    - Dequeue



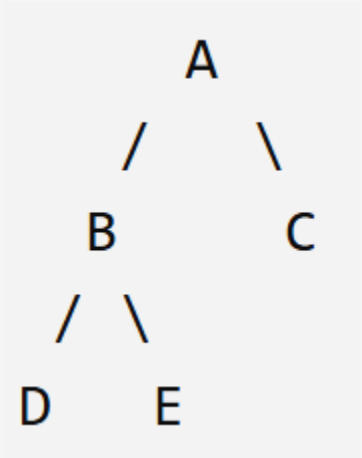
Linear Data Structures

- **Linked List**
  - Set of records linked together by references.
  - Records are often called as nodes and references are called as links or pointers.
  - Operations
    - Insert
    - Delete

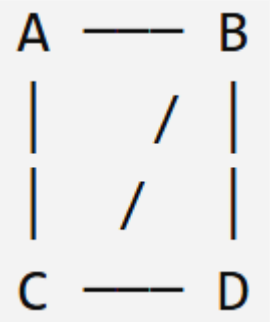


Non-Linear Data Structures

- A data structure is **non-linear** when:
  - all the data elements are not arranged in a linear or sequential order.
  - one element is connected to n number of elements.
  - Example: Trees

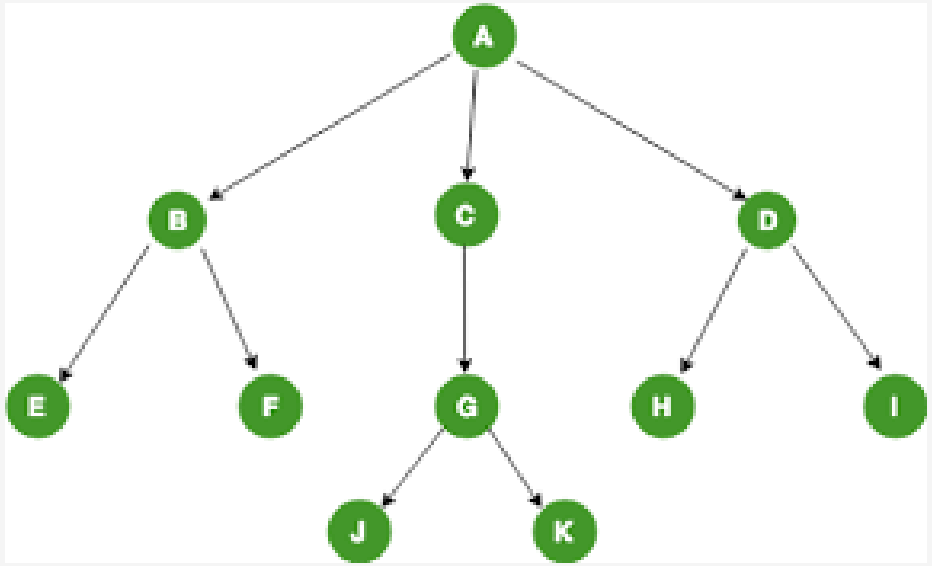


Graphs



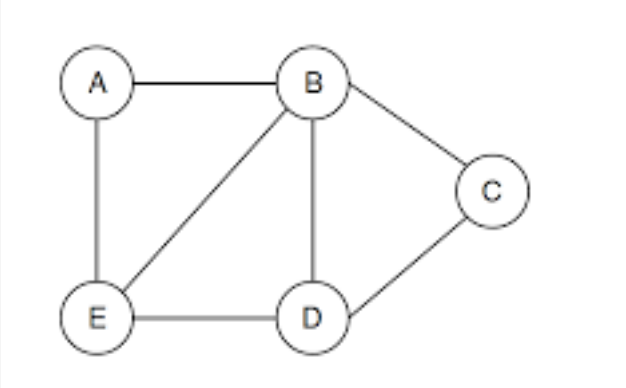
Non-Linear Data Structures

- **Tree**
  - A tree consists of a finite set of elements, called nodes or vertices and a finite set of directed lines, called arcs, that connect pairs of the nodes.
  - Operations
    - Insert
    - Delete
    - Traversal



Non-Linear Data Structures

- **Graph**
  - A graph data structure is a non-linear data structure composed of **Vertices (Nodes)** and **Edges (Links)** that connect those vertices.
  - Depending on the nature of the connections, graphs may be **directed or undirected**.



Real-life Example of Data Structure

- **Array**
  - Online ticket booking system
- **Stack**
  - Working of a browser: implementing redo and undo operations. Example: powerpoint/ microsoft word
- **Queue**
  - Bank, restaurant
- **Linked List**
  - Music player in phones
- **Tree**
  - the file system on a computer stores hierarchical data, like folder structure, XML/HTML data
- **Graph**
  - Google Map/ Road Network
  - storing the friends information on a social networking site

Operations on Data Structures

- **Traversing**: Visiting each element of a data structure systematically
- **Searching**: Finding a particular element
- **Sorting:** Arranging data according to a particular order
- **Inserting**: Adding a new node/record to the structure
- **Deleting**: Removing a node/record from the structure

Introduction to Abstract Data Type (ADT)

- An **Abstract Data Type (ADT)** defines:
    - **What data is represented**
    - **What operations can be performed**

  without specifying exactly **how** those operations are implemented.
- ADT provides abstraction i.e. hiding details from the user.
- ADT can be thought of as a black box which hides the inner structure and design from the user.
- Key idea: **WHAT, not HOW**

Introduction to Abstract Data Type (ADT)

Characteristics of ADT

- **Abstraction — Hide the unnecessary details**
  - ADT shows only the important features and operations to the user.
  - The internal working is hidden. i.e. it concentrates on what the data structure does rather than how it works.
- **Encapsulation — Keep data and operations together**
  - Data and the operations that work on that data are grouped together.
  - Direct access to the internal data is restricted and the user interacts through the defined operations.
- **Information Hiding — Hide implementation**
  - The internal representation of the ADT is hidden from the user.
  - The user does not need to know how an operation is implemented.
- **Implementation Independence — Same ADT, different implementations**
  - An ADT can be implemented in different ways without changing its basic operations.
- **Well-Defined Operations**
  - An ADT specifies what operations are available and what each operation does.
- **Modularity — *Makes programs easier to manage***
  - An ADT can be designed as a separate module so that other parts of the program can use it without knowing its internal implementation.

Introduction to Abstract Data Type (ADT)

Example: ATM

- User knows:
  - Withdraw
  - Deposit
  - Check balance
  - Transfer

- User does **not** need to know the internal implementation.
  Therefore:
  Interface/behavior → WHAT
  Implementation → HOW

Introduction to Abstract Data Type (ADT)

- **Example: Stack as an ADT**:
  - A Stack ADT can specify operations such as:
    - Push() – insert an element into the stack
    - Pop() – delete an element from top of the stack
    - isEmpty() – checks if stack is empty
    - isFull() – checks if stack is full
  - It tells us **what operations** are available.
  - It does not necessarily specify whether the stack must be implemented using:

Array or Linked List.

ADT vs Data Structure

ADT	Data Structure
Defines what operations are available	Defines how data is organized/implemented
Focuses on behavior	Focuses on implementation
Implementation independent	Implementation dependent
Example: Stack ADT	Array-based Stack
Example: Queue ADT	Linked-list Queue

